STUDY REFERENCE — NOT FOR SUBMISSION

PharmaChain Integrity: What It Is, How It Works, Why It's Built This Way

A working reference for explaining this project to anyone — a TTO officer, a faculty evaluator, a patent attorney, a curious classmate — without needing to look anything up. Read it once top to bottom, then use the Q&A section to pressure-test yourself.

CONTENTS

1. The one-paragraph pitch
2. The problem it solves
3. System architecture
4. Tech stack, layer by layer
5. How a record actually moves through it
6. The five novel elements, plainly
7. Key design decisions, and why
8. Anticipated questions

1 The one-paragraph pitch

> PharmaChain Integrity is a system that makes pharmaceutical manufacturing records tamper-evident — not just the data itself, but the *human review process* that approved it. It uses a blockchain smart contract as a write-once ledger that no single person, not even a system administrator with full database access, can silently alter. Every anchor (every "this is now permanently true") requires independent confirmation from more than one party, those parties' signing keys are held by genuinely separate, independently-secured systems, and the mechanism is proven to generalize (M-of-N attestors, arbitrary data types) rather than being a narrow one-off trick.

2 The problem it solves

Pharmaceutical manufacturing is regulated under GxP (Good Manufacturing/Laboratory/Clinical Practice) rules, which require that batch records — the paper or digital trail proving a drug was made correctly — be complete, accurate, and unaltered. In practice, most systems store this in a regular database, which means:

- Anyone with sufficient database access (an insider, a compromised admin account, an attacker who breaches the system) can edit historical records and no one can prove it happened.
- Even if the *data* is protected somehow, the record of *who reviewed it and when* — the approval trail — is usually just more mutable database rows, equally easy to falsify.
- Existing blockchain-based pharma systems (the closest being MediLedger, live since 2017) solve a *different* problem: tracking custody and ownership of physical packaged goods as they move through a supply chain. They don't touch manufacturing-record content or the review process at all.

This project closes that specific, unaddressed gap: it makes both the manufacturing record *and* the fact that it was genuinely, correctly reviewed into cryptographically provable facts, verifiable by anyone, including someone who doesn't trust the database at all.

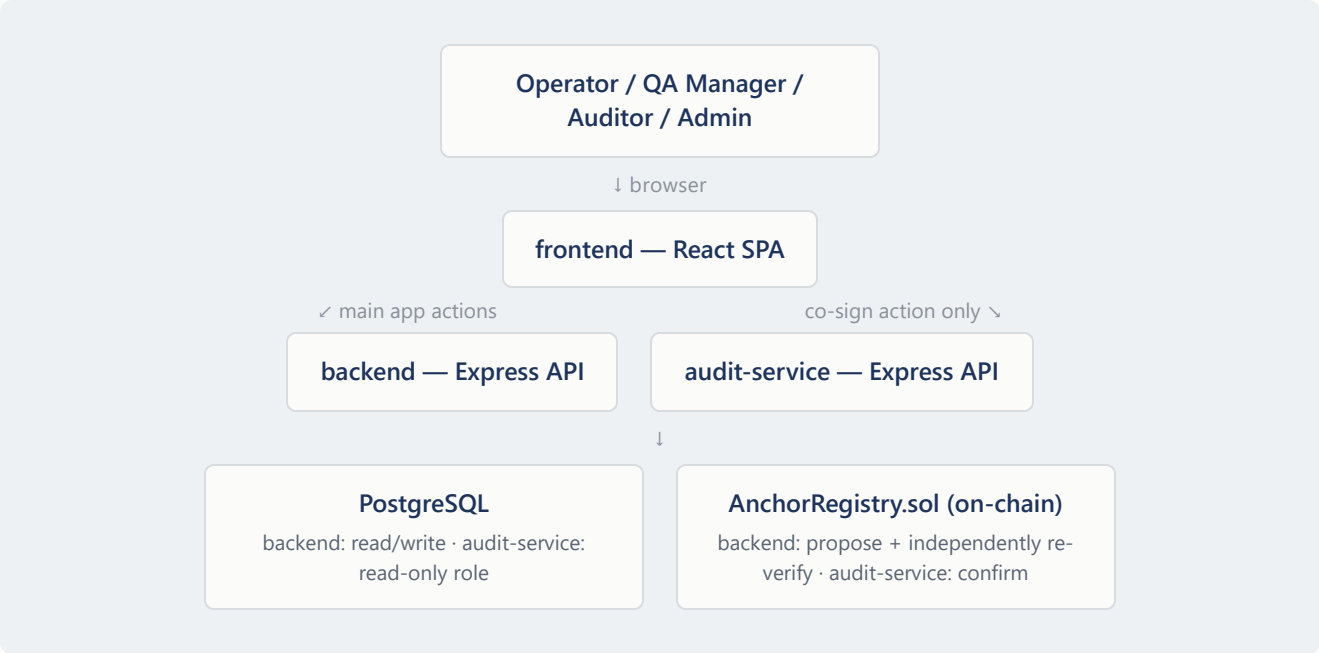
3 System architecture — four independent pieces

The system is a monorepo (one repository, several independently-runnable programs inside it) with four parts:

- **`contracts/`** — one Solidity smart contract, `AnchorRegistry.sol`, deployed to an Ethereum-compatible blockchain. This is the tamper-evident ledger itself. It stores nothing but cryptographic hashes — never actual manufacturing data.
- **`backend/`** — the main application server. Runs the entire manufacturing workflow (batches, records, equipment, users, roles),

talks to a PostgreSQL database, and holds one of the two keys needed to write to the blockchain (the "QA attestor" key).

- `audit-service/` — a second, deliberately separate server. Its only job is independently confirming anchors using the other required key (the "Auditor attestor" key). It has its own login system and only read-only database access — it physically cannot write to the database even if compromised.
- `frontend/` — the web application people actually use, talking to both `backend/` and `audit-service/` depending on the action.



Notice there is no line from `audit-service` to `backend` that says "trust me" — the only connection between them is the blockchain itself. That's deliberate: see §7.

4 Tech stack, layer by layer

contracts/ — the blockchain layer

TECHNOLOGY	ROLE
`Solidity 0.8.24`	The smart contract language — code that runs identically and verifiably on every node of the blockchain network.
`Foundry (forge)`	Fast test framework written for Solidity itself — used for all 51 contract tests, including property-based fuzz tests.

TECHNOLOGY	ROLE
`Hardhat`	Local development blockchain node + TypeScript deployment scripting — runs the actual chain this prototype talks to.
`OpenZeppelin Contracts`	Audited, standard building blocks (e.g. `Ownable` for owner-only functions) instead of reinventing well-known primitives insecurely.

backend/ and audit-service/ — the application layer

TECHNOLOGY	ROLE
`Node.js + TypeScript`	JavaScript runtime with static types — catches a whole class of bugs before code ever runs.
`Express`	Minimal, well-understood web server framework for the HTTP API.
`PostgreSQL`	Relational database — chosen over a NoSQL store because manufacturing records have real relationships (a record belongs to a batch, uses equipment, has an event history) that benefit from foreign keys and transactional guarantees.
`Prisma`	Type-safe database client + migration tool — the database schema and the TypeScript types that read/write it can never silently drift apart.
`ethers.js`	The library both services use to actually sign transactions and talk to the blockchain.
`zod`	Runtime validation of incoming API requests — TypeScript types are erased at runtime, so untrusted input still needs real checking.
`bcrypt + JWT (httpOnly cookies)`	Password hashing and session tokens. httpOnly means the token is invisible to JavaScript in the browser, which blocks a common token-theft attack (XSS).
`Vitest + Supertest`	Test runner and HTTP-request testing — 42 backend tests, 17 audit-service tests.

frontend/ — what people actually use

TECHNOLOGY	ROLE
`React + TypeScript`	Component-based UI library — the industry-standard choice, wide familiarity for anyone reviewing the code.
`Vite`	Build tool / dev server — fast rebuilds during development.
`React Router`	Client-side page navigation without full page reloads.

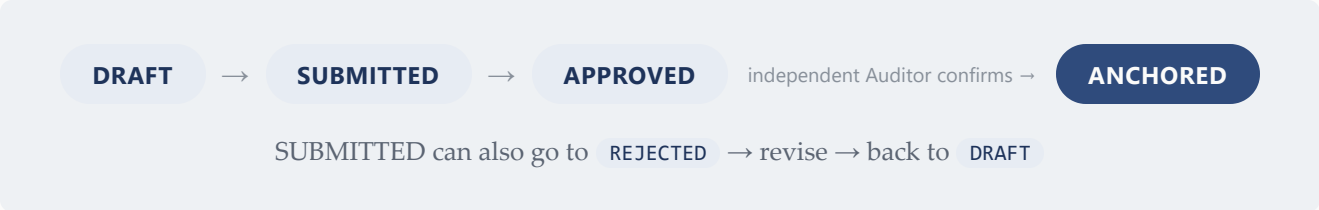
TECHNOLOGY	ROLE
`Plain CSS`	No heavy UI framework — kept deliberately simple since this is a functional prototype demonstrating a mechanism, not a polished consumer product.

5 How a record actually moves through the system

Concrete walkthrough, the way you'd narrate a demo:

1. **An Operator creates a manufacturing record** (e.g., a mixing step for a batch) in `DRAFT` status, then submits it — status becomes `SUBMITTED`.
2. **A QA Manager reviews and approves it.** The system computes a statistical anomaly check first — was this approval unusually fast compared to *that specific reviewer's own history*? If clean, status becomes `APPROVED`, and the backend computes a cryptographic hash of the record and calls the smart contract to *propose* an anchor, using its QA attestor key.
3. **The record is now "pending co-signature"** — proposed, but not yet permanent. A pending-approval badge appears for any Auditor.
4. **A different Auditor reviews it.** They log into `audit-service` (a completely separate login from the main app), which independently recomputes the same hash from its own read-only view of the data and calls the smart contract to *confirm*, using the Auditor key — a key the main backend has never touched.
5. **The main backend checks the blockchain itself** — not audit-service's word for it — confirms the anchor now genuinely exists with the expected hash, and only then marks the record `ANCHORED` in the database.
6. **Anyone, anytime, can verify it.** Recompute the record's hash from current data, compare it to what's on-chain. If someone edited the record afterward (even directly in the database, bypassing the app

entirely), the hashes won't match — tampering is provable, not just suspected.



6 The five novel elements, plainly

- **Per-reviewer anomaly baselining, anchored on-chain.** Instead of one fixed rule ("flag anything under 60 seconds"), each reviewer is judged against their *own* historical average — and the resulting verdict itself is anchored, not just the record, so the fact that something looked suspicious can't be quietly erased.
- **Multi-party propose/confirm attestation.** One party proposes an anchor, a genuinely different party must confirm it, enforced by the smart contract itself — no single key can anchor anything alone.
- **Equipment-state anchoring, same mechanism, zero contract changes.** Equipment calibration events go through the identical propose/confirm flow — proof the core primitive is general-purpose, not a one-off hack for records specifically.
- **Decentralized attestor-set governance.** Even deciding *who is allowed to be an attestor* isn't controlled by one owner key — adding one needs two existing attestors to agree; removing one needs the owner and an attestor to agree.
- **Independent key-custody split + configurable M-of-N.** The two attestor keys live in two separately-secured, separately-authenticated services with no shared secret — and the "how many parties must confirm" threshold is configurable (not hardcoded at two), proven with a real 3-attestor deployment.

7 Key design decisions, and why

Why blockchain at all, instead of just a well-audited database?

A database audit log is still just more rows in the same database — anyone with sufficient access to alter the data can alter the log describing it too. A blockchain gives an append-only structure that isn't under any single party's control to rewrite, so tampering becomes cryptographically detectable rather than merely against policy.

Why anchor only a hash, not the actual manufacturing data?

Privacy and practicality. The blockchain never sees formulas, quantities, or supplier identities — only a fingerprint. Anyone with access to the real data can recompute the fingerprint and compare; no one else learns anything from the chain itself.

Why require two parties to confirm instead of one key anchoring everything?

If one key can unilaterally decide what gets written, the "immutable ledger" only reflects what that one party chooses to record — which defeats the purpose. Requiring a second, independent confirmation means no single compromised or dishonest party can control the record of truth.

Why split the two keys into two separate services instead of one backend holding both?

If one process holds both keys, a single successful attack on that one process breaks the entire two-party guarantee at once. Two genuinely separate services, with separate logins and no shared secret, mean an attacker has to compromise *both* independently — a materially harder bar.

Why does the backend re-verify on-chain instead of just trusting audit-service's "it worked" response?

Defense in depth. Even if audit-service were compromised and lied about having confirmed something, the backend won't record it as true unless it independently sees that confirmation actually exists on the blockchain with the exact expected content — a compromised second service can't be used to fabricate history.

Why generalize to M-of-N attestors instead of stopping at two?

Real deployments may want more independent parties (e.g. adding a regulator alongside manufacturer and auditor) without redesigning the system. Proving the mechanism scales — not just asserting it could — is also stronger evidence for a patent claim.

Why is this a private local blockchain right now, not public Ethereum?

This is a prototype, and the value here is the tamper-evidence property among known, semi-trusted parties (a manufacturer, an auditor, possibly a regulator) — not public, permissionless decentralization. A real deployment would likely use a private/permissioned chain among those specific parties, not the public Ethereum network, so gas costs and public-network throughput aren't relevant concerns.

8 Anticipated questions

Q: ISN'T THIS JUST MEDILEDGER WITH EXTRA STEPS?

A: No — MediLedger tracks ownership and custody of physical packaged goods moving through a supply chain. This system never touches custody at all; it makes the manufacturing record's content and its review process tamper-evident, a problem MediLedger doesn't address.

Q: WHAT'S ACTUALLY NOVEL HERE VERSUS KNOWN TECHNIQUES?

A: No single piece is new in isolation — hash-anchoring, statistical anomaly detection, and multi-signature schemes all exist separately. The novel contribution is the specific *combination*, applied to this specific problem: anchoring the review-process verdict (not just data) with per-individual baselining, requiring independent multi-party confirmation for every anchor including changes to who may attest, generalized to configurable M-of-N, with genuinely independent key custody — proven to work together as one system.

Q: IS THIS ACTUALLY DEPLOYED ANYWHERE?

A: It's a fully working, tested prototype — 172 automated tests across all four layers, verified end-to-end multiple times against a live local blockchain — but deliberately not deployed publicly yet, to avoid triggering public-disclosure issues before the patent/ownership process is resolved.

Q: WHAT HAPPENS IF THE BLOCKCHAIN NODE ITSELF GOES DOWN OR IS ATTACKED?

A: For this prototype (a single local development chain), that's a real single point of failure — a known, disclosed limitation. A production deployment would run on a private/consortium chain replicated across multiple nodes controlled by different parties (manufacturer, auditor, possibly a regulator), which is the standard mitigation and consistent with how the two-party trust model already assumes multiple independent stakeholders.

Q: WHO CAN SEE THE MANUFACTURING DATA — IS THIS PUBLIC?

A: No. The blockchain only ever stores hashes, never the actual data. The application (database, UI) is access-controlled by role (Operator, QA Manager, Auditor, Admin), same as any normal enterprise system.

Q: WHAT WOULD IT TAKE TO MAKE THIS PRODUCTION-READY FOR A REAL PHARMACEUTICAL COMPANY?

A: Integration with their existing systems (MES/ERP), deployment on a real private/consortium blockchain network instead of a local dev chain, hardened production infrastructure — especially the audit-service running as a genuinely separate, independently-operated system (possibly run by a third party, not the manufacturer) — and a formal regulatory validation review (e.g. FDA 21 CFR Part 11 electronic-records compliance).

Q: WHY SHOULD A QA REVIEWER BE JUDGED AGAINST THEIR OWN HISTORY INSTEAD OF ONE FIXED RULE FOR EVERYONE?

A: A fixed global threshold produces both false positives (a naturally fast, careful reviewer gets flagged constantly) and false negatives (a normally slow reviewer's unusually fast decision, still "slow" by the global constant, never gets flagged even though it's a real behavioral

outlier for that person). Baselining against each reviewer's own history catches the second case, which a fixed rule structurally cannot.